

Written Questions:

- 1) Constructor chaining using this() and super(), copy constructors
- 2) Understand how objects are stored in memory
- 3) Explore real life scenarios for each type of inheritance
- 4) Try overloading constructors and the need of overloading
- 5) Explore interface-based inheritance vs class-based inheritance, diamond problem solution in Java
- 6) Understand method overriding rules, covariant return types, and @Override annotation
- 7) Explore:
 - i) Built-in functional interfaces: Predicate, Function, Consumer, Supplier, BiFunction, UnaryOperator
 - ii) Stream pipeline optimization and performance considerations
 - iii) Optional class with flatMap(), orElseThrow(), ifPresentOrElse()
 - iv) Method references (static, instance, constructor), effectively final variables in lambdas
 - v) Lazy evaluation in streams, stateless vs stateful operations, short-circuit operations
- 8) Explore different ways to initialize static members of a class
- 9) Explore the significance of String pool in memory management
- 10) Understand String immutability in Java
- 11) Deciding when to use StringBuilder vs StringBuffer
- 12) String interning, string constant pool memory management, and intern() method
- 13) Explore Regular expressions (regex) for pattern matching in strings.
- 14) Understand the difference between throw and throws keyword
- 15) Why are multiple catch blocks used in Java exception handling?
- 16) Understand the benefit of using try with resources block
- 17) Checked vs unchecked exceptions - when to use which
- 18) Deciding when to use which collection based on the requirement
- 19) Understand the internal working of HashMap (hashing, bucket, collision handling)
- 20) Difference between Iterator and ListIterator
- 21) Difference between fail-fast vs fail-safe iterators
- 22) Custom equals() and hashCode() implementation and its impact on collections
- 23) Difference between Comparable and Comparator with multiple sorting criteria
- 24) Difference between HashSet and LinkedHashSet.
- 25) Difference between HashSet and TreeSet.
- 26) Difference between LinkedHashSet and TreeSet.
- 27) How do HashSet and HashMap work internally in Java? What is the role of equals() and hashCode() methods in this context?
- 28) Explore map interface and observe the differences among them
- 29) Explain multithreading using examples. Explain standard examples like a producer- consumer problem.
- 30) How to create new threads? Explain "implements Runnable" vs "extends Thread".
- 31) Why Synchronization is needed?
- 32) When race condition occurs?
- 33) Difference between sleep and wait?
- 34) Explore producer consumer problem?

Code Questions:

- 1) Implement a Student Information System with multiple constructors and demonstrate shallow vs deep copy.
- 2) Create a Shape hierarchy (Shape → 2DShape/3DShape → Circle/Rectangle/Sphere/Cube) with area and volume calculations
- 3) Implement a zoo management system demonstrating IS-A and HAS-A relationships (Composition)
- 4) Create a custom enum for Days of Week with methods (isWeekend(), Weekday())
- 5) Build a text analyzer that counts vowels, consonants, special characters, and generates statistics

- 6) Implement a custom exception hierarchy for an e-commerce application(payment, inventory, shipping exceptions)
- 7) Implement a shopping cart system using appropriate collections (HashMap for products, ArrayList for order history)
- 8) Calculate the factorial of the given number using stream operations.
- 9) Write a program to calculate sum of integers using streams.
- 10) Create an IntStream to represent numbers from 1 to 10. Call various functions like sum(), summaryStatistics() and observe the output
- 11) Write a generic static method to find minimum from an array of Number.
- 12) Write a generic sort method for implementing selection sort algorithm with given comparator. Refer code below.

```
```Java
static <T> void selectionSort(T[] arr, Comparator<T> c) {
 for(int i=0; i<arr.length-1; i++) {
 for(int j=i+1; j<arr.length; j++) {
 if(c.compare(arr[i], arr[j]) > 0) {
 T temp = arr[i];
 arr[i] = arr[j];
 arr[j] = temp;
 }
 }
 }
}
```
```

- 13) Use Arrays.sort() to sort array of Students using Comparator. The 1st level sorting should be on city (desc), 2nd level sorting should be on marks (desc), 3rd level sorting should be on name (asc).

```
```Java
class Student {
 private int roll;
 private String name;
 private String city;
 private double marks;
 // ...
}
```
```